

CSA0617-Design and Analysis of Algorithms

Day 3 Programming

Name: Y.Thanushma

Registration Number:192425355

1. Number of Ways to Move a Ball Out of the Grid

Aim

To find the number of ways to move a ball out of an $m \times n$ grid boundary in exactly N steps using recursion.

Algorithm

1. Read the grid dimensions m and n , number of steps N , and starting position (i,j) .
2. Check whether the ball has moved outside the grid.
 - If yes, return 1.
3. If $N == 0$ and the ball is still inside, return 0.
4. Move the ball in four directions:
 - Up
 - Down
 - Left
 - Right
5. Recursively calculate the number of ways for each direction.
6. Add the four results.
7. Display the total number of ways.

Code:

```
def f(m,n,N,i,j):  
    if i<0 or i>=m or j<0 or j>=n:  
        return 1  
    if N==0:  
        return 0  
    return (f(m,n,N-1,i-1,j)+f(m,n,N-1,i+1,j)+
```

```

        f(m,n,N-1,i,j-1)+f(m,n,N-1,i,j+1))
print(f(2,2,2,0,0))
print(f(1,3,3,0,1))

```

Output:

```

6
12

```

Result:

Thus, the number of ways to move the ball out of the grid boundary in exactly N steps is successfully calculated.

2: House Robber II – Maximum Money in Circular Houses

Problem Statement

You are a professional robber planning to rob houses along a street. Each house contains a certain amount of money. The houses are arranged in a circle, so the first house is adjacent to the last house. Two adjacent houses cannot be robbed on the same night.

Find the maximum amount of money that can be robbed without alerting the police.

Example

Input: nums = [2, 3, 2]

Output: 3

Input: nums = [1, 2, 3, 1]

Output: 4

Aim

To find the maximum amount of money that can be robbed from houses arranged in a circle without robbing two adjacent houses.

Algorithm

1. If there is only one house, return its money.
2. Since the houses are in a circle, the first and last houses cannot both be robbed.

3. Consider two cases:
 - Rob houses from first to second-last.
 - Rob houses from second to last.
4. For each case, use Dynamic Programming:
 - prev stores the maximum money up to the previous house.
 - curr stores the maximum money up to the current house.
5. For every house:
 - Skip it: curr
 - Rob it: prev + money
6. Take the maximum of the two cases.
7. Display the result.

Execution:

```
def rob(a):
    p=c=0
    for x in a:
        p,c=c,max(c,p+x)
    return c

def houseRobber(nums):
    if len(nums)==1:
        return nums[0]
    return max(rob(nums[:-1]), rob(nums[1:]))

print(houseRobber([2,3,2]))
print(houseRobber([1,2,3,1]))
```

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56)
[MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Thanushma/Documents/DAA
/1.py =====
6
12
>>> ===== RESTART: C:/Users/Thanushma/Documents/DAA
/1.py =====
3
4
>>>
```

Result: Thus, the maximum amount of money that can be robbed without alerting the police is successfully calculated.

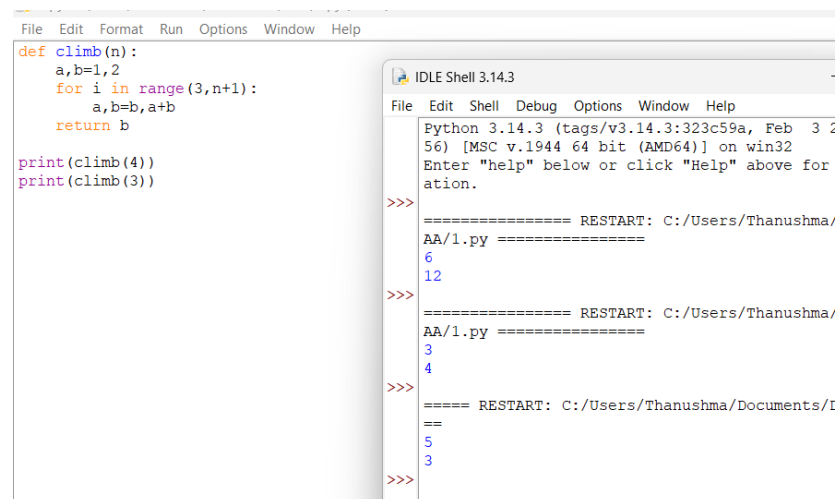
3. Climbing Stairs

Aim: To find the number of distinct ways to climb a staircase of n steps when each move can be either 1 step or 2 steps.

Algorithm:

1. If $n = 1$, return 1.
2. If $n = 2$, return 2.
3. Initialize $a = 1$ and $b = 2$.
4. For each step from 3 to n :
 - Calculate $c = a + b$.
 - Set $a = b$ and $b = c$.
5. Return b .
6. Display the number of ways.

Execution:



```
File Edit Format Run Options Window Help
def climb(n):
    a,b=1,2
    for i in range(3,n+1):
        a,b=b,a+b
    return b
print(climb(4))
print(climb(3))

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2024) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for
ation.
>>>
===== RESTART: C:/Users/Thanushma/
AA/1.py =====
6
12
>>>
===== RESTART: C:/Users/Thanushma/
AA/1.py =====
3
4
>>>
===== RESTART: C:/Users/Thanushma/Documents/I
==
5
3
>>>
```

Result:

Thus, the number of distinct ways to climb the staircase is successfully calculated.

4. Unique Paths in a Grid

Problem Statement

A robot is located at the top-left corner of an $m \times n$ grid. The robot can move only down or right at any point. Find the number of possible unique paths to reach the bottom-right corner.

Examples

Input: $m = 7, n = 3$

Output: 28

Input: $m = 3, n = 2$

Output: 3

Aim

To find the number of unique paths for a robot to travel from the top-left corner to the bottom-right corner of an $m \times n$ grid using Dynamic Programming.

Algorithm

1. Create a DP array of size n and initialize all values to 1.
2. Traverse the grid from the second row.
3. For each column from the second column:
 - Add the value from the left to the current value.
4. The final value represents the total number of unique paths.
5. Display the result.

Code:

```
def paths(m,n):  
    a=[1]*n  
    for i in range(1,m):  
        for j in range(1,n):  
            a[j]+=a[j-1]  
    return a[-1]
```

```
print(paths(7,3))
```

```
print(paths(3,2))
```

Output:

```
==== RESTART: C:/Users/Thanushma/Documents/DAA/1.py ====
28
3
|
```

Result:

Thus, the number of unique paths from the top-left corner to the bottom-right corner of the grid is successfully calculated.

5.Positions of Large Groups

Aim: To find and display the start and end indices of all large groups containing three or more consecutive identical characters in a string.

Algorithm

1. Start from index 0.
2. Find consecutive characters that are the same.
3. Store the starting index and ending index of the group.
4. If the group length is 3 or more, add [start, end] to the result.
5. Continue until the end of the string.
6. Display all large group intervals.

Code:

```
1.py - C:/Users/Thanushma/Documents/DAA/1.py (3.14.3)
File Edit Format Run Options Window Help
def large(s):
    r=[]; i=0
    while i<len(s):
        j=i
        while j<len(s) and s[j]==s[i]:
            j+=1
        if j-i>=3:
            r.append([i,j-1])
        i=j
    return r
print(large("abbxxxxzzy"))
print(large("abc"))
```

Output:

```
----- RESIDENT: C:/Users/Manishma/Documents/BAH/1.py -----  
[[3, 6]]  
[]
```

Result:

Thus, all large groups containing 3 or more consecutive identical characters and their corresponding intervals are successfully identified.

Question 6: Conway's Game of Life

Problem Statement

The Game of Life is a cellular automaton devised by mathematician John Horton Conway. The board consists of an $m \times n$ grid where:

- 1 represents a live cell.
- 0 represents a dead cell.

Each cell interacts with its 8 neighboring cells.

Rules

1. A live cell with fewer than 2 live neighbors dies.
2. A live cell with 2 or 3 live neighbors survives.
3. A live cell with more than 3 live neighbors dies.
4. A dead cell with exactly 3 live neighbors becomes alive.

All cells are updated simultaneously.

Aim: To generate the next state of a Game of Life board by applying Conway's four rules to every cell.

Algorithm

1. Read the $m \times n$ board.
2. For every cell, count the number of live neighbors among its 8 surrounding cells.

3. Apply the rules:

- Live + fewer than 2 neighbors → Dead.
- Live + 2 or 3 neighbors → Live.
- Live + more than 3 neighbors → Dead.
- Dead + exactly 3 neighbors → Live.

4. Store the new state separately so that changes do not affect other cells.

5. Display the updated board.

Code:

```
1.py - C:/Users/Thanushma/Documents/DAA/1.py (3.14.3)
File Edit Format Run Options Window Help
def life(b):
    m,n=len(b),len(b[0])
    r=[[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            c=sum(b[x][y] for x in range(max(0,i-1),min(m,i+2))
                  for y in range(max(0,j-1),min(n,j+2))
                  if (x,y) != (i,j))
            r[i][j]=1 if c==3 or (b[i][j] and c==2) else 0
    return r

print(life([[0,1,0],[0,0,1],[1,1,1],[0,0,0]]))
print(life([[1,1],[1,0]]))
|
```

Output:

```
| [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
| [[1, 1], [1, 1]]
```

Result:

Thus, the next generation of the Game of Life board is successfully calculated by applying Conway's rules to all cells simultaneously.

Question 7: Champagne Tower

Problem Statement

Champagne glasses are arranged in a pyramid. The first row contains 1 glass, the second row contains 2 glasses, and so on up to the 100th row. Each glass can hold 1 cup of champagne.

When a glass becomes full, any excess champagne flows equally into the two glasses directly below it.

Given the number of cups poured, find how full the glass at query_row and query_glass is. Both indices are 0-based.

Examples

Example 1:

Input:

poured = 1

query_row = 1

query_glass = 1

Output:

0.00000

Example 2:

Input:

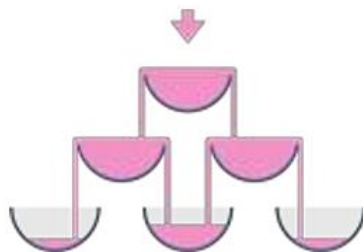
poured = 2

query_row = 1

query_glass = 1

Output:

0.50000



Aim: To determine the amount of champagne present in a specific glass of a champagne pyramid after pouring a given number of cups.

Algorithm

1. Create a 2D array to represent the champagne tower.
2. Pour the given amount of champagne into the top glass.
3. For every glass:
 - If it contains more than 1 cup, calculate the excess.
 - Divide the excess equally between the two glasses below it.
4. Continue until the required row is reached.

5. The amount in the requested glass cannot exceed 1 cup.
6. Display the result.

Code:

```
1.py - C:/Users/Thanushma/Documents/DAA/1.py (3.14.3)
File Edit Format Run Options Window Help
def champagne(p,r,g):
    a=[[0]*101 for _ in range(101)]
    a[0][0]=p
    for i in range(r):
        for j in range(i+1):
            if a[i][j]>1:
                x=(a[i][j]-1)/2
                a[i+1][j]+=x
                a[i+1][j+1]+=x
    return min(1,a[r][g])

print(f"{champagne(1,1,1):.5f}")
print(f"{champagne(2,1,1):.5f}")
```

Output:

```
> 0.00000
  0.50000
```

Result:

Thus, the amount of champagne in the specified glass is successfully calculated.